

CSE 60745 — Graph Learning: Techniques & Applications

Hands-on HW2 — Part (B): Graph Embedding (30 pts)

1 Task 3 — Graph Embedding over Homogeneous Structures

1.1 Approach

All four notebook TODOs are implemented. Every provided helper — data loading, the cached adjacency lookups, the whole evaluation toolset of section 5 and `run_experiment` — is used verbatim, with `RANDOM_SEED = 0`. Loading reproduces the published statistics exactly: 7,624 users, 19,464 observed edges, 666 isolated users, mean degree 5.106, max degree 153, 734 components, and $4,171 \times 21$ validation/test queries.

sample_walks. A strategy is nothing but a transition rule; the *walk budget is identical for every strategy* (10 walks of length 40 from each of the 6,958 non-isolated users, giving 69,580 walks in every run). The 666 isolated users return `[]`, as the docstring permits, and therefore keep a zero embedding row. Five transition rules are compared, giving the seven configurations of Table 1 (node2vec is run at three (p, q) settings):

strategy	transition rule $P(x \mid \cdot)$	hypothesis being tested
DeepWalk	uniform over $N(v)$	baseline: unbiased 1st-order walk
node2vec	$\propto 1/p, 1, 1/q$ by distance to t	$q < 1$ outward (homophily), $q > 1$ inward (structural equivalence)
RWR	uniform, teleport to start w.p. r	a strongly <i>personalised</i> context should sharpen pairwise proximity, at the cost of global community signal
deg-corrected	$\propto \deg(x)^{-\alpha}$	uniform walks over-visit hubs; equalising visit counts should lift the low-degree source buckets
triadic	$\propto 1 + N(v) \cap N(x) $	preferring triangle-closing steps keeps the walk inside a community, i.e. maximal homophily

RWR, deg-corrected and triadic are my own additions. node2vec uses the standard $O(1)$ rejection sampler rather than materialising alias tables, which keeps a second-order walk at 1.3s for the whole corpus.

train_embedding. Skip-Gram with negative sampling (`gensim.Word2Vec, sg=1`) over the walk corpus. I set `sample=0`, i.e. *no* frequent-token subsampling: subsampling would itself down-weight hubs and so half-implement the deg-corrected strategy, confounding that comparison. `workers=1` makes every run bit-exactly reproducible (gensim's multi-threaded training is not deterministic).

Downstream models (held fixed). As the notebook demands, the link predictor and the country classifier are the very same code for every row of the comparison table; the embedding is the only thing that changes. Both ℓ_2 -normalise Z row-wise (zero rows stay zero), standardise, and fit a logistic regression. A pair (u, v) becomes

$$\phi(u, v) = \left[\underbrace{z_u \odot z_v}_{\text{Hadamard}}, \underbrace{|z_u - z_v|}_{\text{abs. diff.}}, \cos(z_u, z_v), z_u^\top z_v, \|z_u - z_v\|_2 \right] \in \mathbb{R}^{2d+3},$$

the node2vec paper’s two best binary operators concatenated with three scalar similarities. `score` returns the continuous `decision_function`, so higher always means ”more likely an edge”.

Fixed hyper-parameters (identical in every run).

walks per node	10	embedding dim. d	128
walk length	40	Skip-Gram window	5
corpus size	69,580 walks	negative samples	5
seed	0 (also 1, 2)	epochs	5
LP regulariser	$C \in \{0.1, 1, 10\}$, chosen by val. MRR	subsampling	off (<code>sample=0</code>)
CC regulariser	$C \in \{0.1, 1, 10, 100\}$, chosen by val. micro-F1	solver	lbfgs

1.2 Implementation

Only the code I wrote is listed; the provided scaffolding is omitted.

```
def sample_walks(self, G, start):
    """'num_walks' walks of length 'walk_length', all beginning at 'start'."""
    adj, deg = adjacency_lookups(G)
    if deg[start] == 0: # isolated node -> no walks (allowed)
        return []
    rng, L, R, strat = self._rng, self.walk_length, self.num_walks, self.strategy

    if strat == "node2vec": # 2nd-order, O(1) rejection sampling
        NBR = self._nbr_lists(G) # cached sorted neighbour lists
        p, q = self.params["p"], self.params["q"]
        inv_p, inv_q, wmax = 1/p, 1/q, max(1/p, 1.0, 1/q)
        walks = []
        for _ in range(R):
            walk = [start, rng.choice(NBR[start])] # 1st step unbiased
            while len(walk) < L:
                cur, prev = walk[-1], walk[-2]
                cn, prev_nbrs = NBR[cur], adj[prev]
                while True:
                    x = rng.choice(cn)
                    w = inv_p if x == prev else (1.0 if x in prev_nbrs else inv_q)
                    if rng.random() * wmax < w:
                        break
                walk.append(x)
            walks.append(walk)
        return walks

    if strat in ("degcorr", "triadic"): # 1st-order, edge-weighted
        tables = self._weighted_tables(G) # per node: (nbrs, cumulative weights)
        walks = [] # degcorr: w(x) = deg(x)**-alpha
        for _ in range(R): # triadic: w(x) = 1 + |N(cur) & N(x)|
            walk = [start]
            while len(walk) < L:
                nbrs, cum = tables[walk[-1]]
                walk.append(rng.choices(nbrs, cum_weights=cum, k=1)[0])
            walks.append(walk)
        return walks
```

```

NBR = self._nbr_lists(G) # "uniform" (DeepWalk) and "rwr"
restart = self.params.get("restart", 0.0) if strat == "rwr" else 0.0
walks = []
for _ in range(R):
    walk = [start]
    while len(walk) < L:
        if restart and rng.random() < restart:
            walk.append(start) # teleport home, then carry on
        else:
            walk.append(rng.choice(NBR[walk[-1]]))
    walks.append(walk)
return walks

```

Listing 1: TODO 1 — sample_walks

```

def train_embedding(self, walks, n_nodes):
    d = self.params.get("dim", 128)
    model = Word2Vec(sentences=[[str(v) for v in w] for w in walks],
                     vector_size=d, window=self.params.get("window", 5),
                     sg=1, hs=0, negative=self.params.get("negative", 5),
                     epochs=self.params.get("epochs", 5),
                     min_count=0, sample=0, seed=self.seed, workers=1)
    Z = np.zeros((n_nodes, d), dtype=float) # nodes never walked keep a zero row
    Z[[int(k) for k in model.wv.index_to_key]] = model.wv.vectors
    return Z

```

Listing 2: TODO 2 — train_embedding

```

def _unit(Z): # row-wise L2 norm; all-zero rows stay zero
    n = np.linalg.norm(Z, axis=1, keepdims=True)
    return Z / np.where(n == 0.0, 1.0, n)

def _pair_features(Z, pairs): # (n_q, n_cand, 2) -> (n_q*n_cand, 2d+3)
    u, v = Z[pairs[:, 0]].reshape(-1, Z.shape[1]), Z[pairs[:, 1]].reshape(-1, Z.shape[1])
    had, dif = u * v, np.abs(u - v)
    dot = had.sum(1, keepdims=True)
    l2 = np.linalg.norm(u - v, axis=1, keepdims=True)
    cos = dot / np.maximum(np.linalg.norm(u, axis=1, keepdims=True)
                          * np.linalg.norm(v, axis=1, keepdims=True), 1e-12)
    return np.hstack([had, dif, cos, dot, l2])

class LinkPredictor:
    C_GRID = (0.1, 1.0, 10.0)

    def fit(self, Z, pairs_train, y_train, pairs_val, y_val):
        Zn = _unit(np.asarray(Z, dtype=float))
        Xtr = _pair_features(Zn, pairs_train)
        self.scaler = StandardScaler().fit(Xtr)
        Xtr, ytr = self.scaler.transform(Xtr), np.asarray(y_train).ravel()
        Xva = self.scaler.transform(_pair_features(Zn, pairs_val))
        nq, nc = pairs_val.shape[:2]
        pos_val = np.argmax(np.asarray(y_val), axis=1)

        best = (-1.0, None, None)
        for C in self.C_GRID: # model selection on validation MRR
            clf = LogisticRegression(C=C, max_iter=2000, random_state=RANDOM_SEED)
            clf.fit(Xtr, ytr)
            s = clf.decision_function(Xva).reshape(nq, nc)
            rank = 1 + (s > s[np.arange(nq), pos_val][:, None]).sum(1)
            mrr = float(np.mean(1.0 / rank))
            if mrr > best[0]:
                best = (mrr, clf, C)
        self.val_mrr_, self.clf, self.C_ = best
        return self

    def score(self, Z, pairs): # continuous, higher = more likely
        X = self.scaler.transform(_pair_features(_unit(np.asarray(Z, float)), pairs))

```

```
return self.clf.decision_function(X).reshape(pairs.shape[:2])
```

Listing 3: TODO 3a — LinkPredictor (and the pair feature map)

```
class CountryClassifier:
    C_GRID = (0.1, 1.0, 10.0, 100.0)

    def fit(self, Z_train, y_train, Z_val, y_val):
        Xtr = _unit(np.asarray(Z_train, dtype=float))
        self.scaler = StandardScaler().fit(Xtr)
        Xtr = self.scaler.transform(Xtr)
        Xva = self.scaler.transform(_unit(np.asarray(Z_val, dtype=float)))
        best = (-1.0, None, None)
        for C in self.C_GRID: # selection on validation micro-F1
            clf = LogisticRegression(C=C, max_iter=5000, random_state=RANDOM_SEED)
            clf.fit(Xtr, y_train)
            acc = float((clf.predict(Xva) == np.asarray(y_val)).mean())
            if acc > best[0]:
                best = (acc, clf, C)
        self.val_micro_f1_, self.clf, self.C_ = best
        return self

    def predict(self, Z):
        return self.clf.predict(self.scaler.transform(_unit(np.asarray(Z, float))))
```

Listing 4: TODO 3b — CountryClassifier

1.3 Results

Every number below was measured on this machine. Because the seven strategies turned out to lie close together, I ran the *whole* protocol three times (seeds 0,1,2; the seed drives both the walk sampling and the Skip-Gram initialisation) and report the mean, so that a difference can be compared against the run-to-run noise. Table 1 is `comparison.table()` averaged over those three runs, with one added column (validation macro-F1, which `run_experiment` does not itself record).

Table 1: The scalar metrics of `comparison.table()` together with a validation macro-F1 column that `run_experiment` does not record, mean over seeds {0,1,2}; its six `val_hit@1_deg*` columns are given in full in Table 3. Best per column in bold. The last two rows give the mean within-strategy standard deviation over the three seeds — the noise floor a difference must beat — and the best-minus-worst spread expressed in those units. Columns marked † are measured on the split that `fit` selects *C* on and are therefore optimistic; the test columns are the ones that carry the argument.

strategy	friend recommendation				country classification			
	val H@1 [†]	val MRR [†]	test H@1	test MRR	val mi-F1 [†]	val ma-F1	test mi-F1	test ma-F1
DeepWalk	0.6040	0.7227	0.6009	0.7207	0.7641	0.6639	0.7629	0.6833
n2v $p=1, q=0.5$	0.5995	0.7202	0.5959	0.7184	0.7636	0.6621	0.7621	0.6825
n2v $p=1, q=2$	0.6027	0.7210	0.6011	0.7196	0.7619	0.6467	0.7607	0.6802
n2v $p=q=0.25$	0.6039	0.7217	0.6061	0.7228	0.7610	0.6581	0.7628	0.6796
RWR $r=0.15$	0.5870	0.7119	0.5889	0.7124	0.7637	0.6521	0.7620	0.6733
deg-corr. $\alpha=1$	0.6003	0.7144	0.5990	0.7121	0.7575	0.6421	0.7607	0.6781
triadic	0.6066	0.7247	0.6087	0.7248	0.7645	0.6833	0.7661	0.6974
<i>noise floor</i>	0.0028	0.0021	0.0048	0.0031	0.0055	0.0077	0.0032	0.0062
<i>spread / noise</i>	7.0×	6.0×	4.1×	4.2×	1.3×	5.4×	1.7×	3.9×

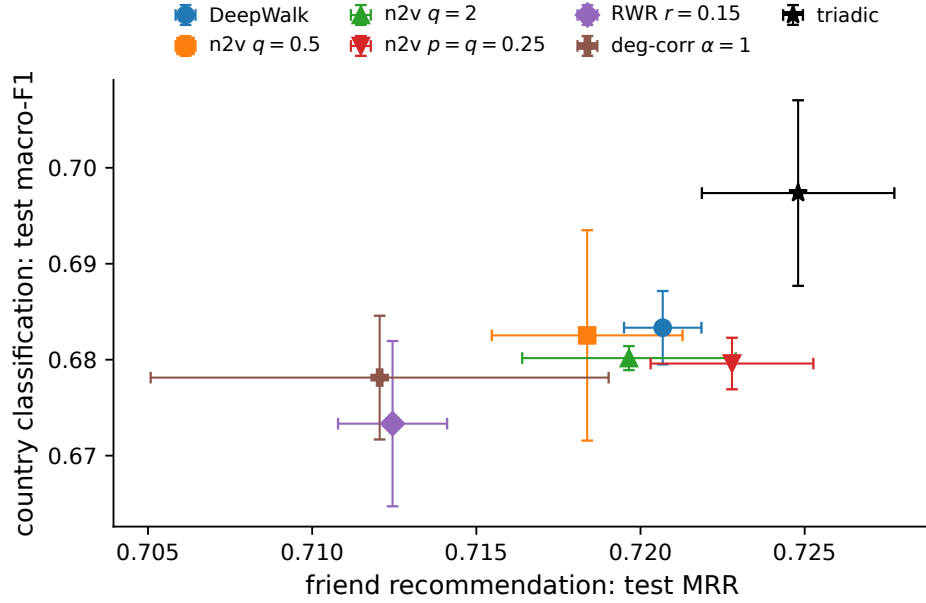


Figure 1: Each strategy on both tasks at once; error bars are ± 1 sd over the three seeds. There is no trade-off frontier — the triadic walk sits alone in the top-right corner.

Per-source-degree breakdown. Table 2 gives `breakdown_by_source_degree` on the test split for the baseline, the best strategy and the worst, with both metrics; Figure 2 plots the Hit@1 half of it with seed error bars. Table 3 then gives the validation Hit@1 breakdown for *all seven* strategies — these are literally the six `val_hit@1_deg*` columns that `run_experiment` records, which is why they are on the validation split. Note that triadic, which wins every aggregate column of Table 1, wins only three of the six degree buckets there: node2vec $p=q=0.25$ takes the cold and 8–15 buckets and $q=2$ ties the 2–3 bucket. Per-bucket margins are small and the buckets are 3–20 $\times$ smaller than the full split, so individual cells are much noisier than the aggregates; the consistent pattern is the one visible in the two highest-degree buckets, not any single cell.

Table 2: `breakdown_by_source_degree`, test split, mean over seeds $\{0, 1, 2\}$, by number of friends the *source* user has in G_{train} : the baseline, the best strategy and the worst.

source degree	n	DeepWalk (baseline)		triadic (best)		RWR (worst)	
		Hit@1	MRR	Hit@1	MRR	Hit@1	MRR
0 (cold)	216	0.0447	0.1887	0.0293	0.1759	0.0447	0.1825
1	266	0.4386	0.5672	0.4423	0.5680	0.4499	0.5705
2–3	469	0.5615	0.6823	0.5672	0.6852	0.5529	0.6752
4–7	842	0.6675	0.7709	0.6710	0.7732	0.6524	0.7616
8–15	939	0.6784	0.7905	0.6904	0.7959	0.6730	0.7853
≥ 16	1439	0.6377	0.7665	0.6502	0.7744	0.6159	0.7541

Table 3: The six `val_hit@1_deg*` columns that `run_experiment` records, for all seven strategies: validation Hit@1 split by source degree, mean over seeds $\{0, 1, 2\}$. Best per bucket in bold. Bucket sizes are 213, 261, 524, 842, 973, 1358 queries.

strategy	0 (cold)	1	2-3	4-7	8-15	≥ 16
DeepWalk (uniform)	0.0594	0.4751	0.5731	0.6457	0.6739	0.6502
node2vec $p=1, q=0.5$	0.0516	0.4687	0.5649	0.6382	0.6691	0.6502
node2vec $p=1, q=2$	0.0626	0.4674	0.5731	0.6394	0.6714	0.6529
node2vec $p=q=0.25$	0.0673	0.4713	0.5681	0.6441	0.6804	0.6478
RWR $r=0.15$	0.0579	0.4841	0.5407	0.6314	0.6564	0.6303
deg-corrected $\alpha=1$	0.0563	0.4559	0.5439	0.6393	0.6762	0.6564
triadic	0.0501	0.4879	0.5681	0.6461	0.6752	0.6581

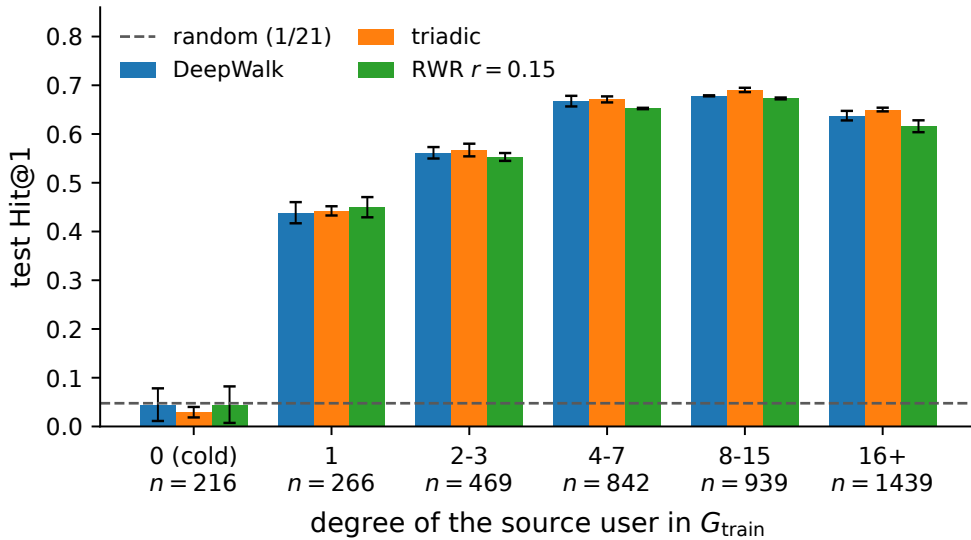


Figure 2: Test Hit@1 by source degree for the baseline (DeepWalk), the best strategy (triadic) and the worst (RWR); error bars are ± 1 sd over seeds $\{0, 1, 2\}$. The cold-start bucket sits at chance for every strategy.

Control. To check that the evaluation path itself carries no signal, I ran the identical pipeline on a random Gaussian Z : it gives test Hit@1 0.0671 and MRR 0.2009 (chance is $1/21 = 0.0476$ and 0.1736) and country test micro-F1 0.1340, far below the majority-class rate of 0.2062. The learned embeddings are therefore genuinely doing the work.

1.4 Analysis

The same embedding wins both tasks. The triadic walk is best in *every* column of Table 1, and Figure 1 shows it alone in the top-right corner: there is no homophily-versus-structural-equivalence trade-off to negotiate on this dataset. That is not an accident of the metrics — both downstream tasks on this graph are homophily tasks. Friends of a user are found inside their community, and country is an extremely assortative label on a social network (people mostly follow people in their own country), so an embedding that sharpens community boundaries helps the

link predictor and the country classifier for the same reason. A dataset where node labels were a function of *role* rather than community (bridges, hubs, peripherals) is where one would expect the two tasks to pull apart, and where a $q > 1$ walk should have paid off.

Which differences are real. Most of them are small, so the noise floor matters — and it has to be read off the *test* columns. `LinkPredictor.fit` selects C on validation MRR and `CountryClassifier.fit` selects C on validation accuracy, so val H@1, val MRR and val micro-F1 are measured on the very split that chose the model and their large ratios ($7.0\times$, $6.0\times$, $1.3\times$) are optimistic. I therefore rest the argument on the held-out columns: the between-strategy spread is $4.1\times$ the within-strategy sd for test Hit@1, $4.2\times$ for test MRR and $3.9\times$ for test macro-F1. Those are real effects on data no model-selection decision ever saw.

But for country *micro-F1* the test spread is only $1.7\times$ the noise (0.7607–0.7661): **on micro-F1 the seven strategies are statistically indistinguishable.** Anyone reading only the micro-F1 column would conclude that the sampling strategy does not matter; the macro-F1 columns show that it does, and by a wide margin. Test macro-F1 spreads 0.0240 ($3.9\times$), and validation macro-F1 spreads 0.0411 — $5.4\times$ its noise floor, the second-largest effect anywhere in this study, with triadic at 0.6833 against deg-corrected’s 0.6421. Validation macro-F1 deserves a little more trust than the other val columns, because C is selected on validation *accuracy*, not macro-F1 — macro-F1 is not the selection objective, merely measured on the selection split — and the unbiased test macro-F1 column agrees with it on both the winner and the ordering.

This is exactly the imbalance effect the question points at: the label distribution runs from 1,572 users in the largest country to 16 in the smallest (only 2 of which land in the 763-user training split), so micro-F1 is dominated by a handful of big classes that every embedding already gets right, and only macro-F1 is sensitive to what happens in the tail. Choosing a sampling strategy on micro-F1 would be choosing on noise.

node2vec buys nothing here. All three (p, q) settings sit within noise of plain DeepWalk (test MRR 0.7184, 0.7196, 0.7228 versus 0.7207), and the homophily-leaning setting $q=0.5$ is if anything the *worst* of the three. I read this as a property of the graph rather than of node2vec: with mean degree 5.1 and 734 components, a length-40 uniform walk is already confined to a small neighbourhood, so re-weighting second-order transitions has little room to change which nodes co-occur. The second-order machinery costs a doubling of sampling time and returns nothing measurable.

Why triadic works and the other two ideas do not. The triadic rule, $P(x) \propto 1 + |N(v) \cap N(x)|$, prefers steps that close a triangle. It is the only one of my three ideas that adds *new information* — it looks at the graph two hops out, whereas uniform, RWR and degree-corrected walks only ever consult the current node’s neighbour list. That extra information is precisely a community signal, which is why its largest gains are on country macro-F1: +0.0194 on validation (0.6833 vs 0.6639, 2.5 noise-sds) and +0.0141 on test (0.6974 vs 0.6833, 2.3 noise-sds). Macro-F1 is the metric that rewards getting the *small* countries right, and a small country is exactly the kind of tight, triangle-dense community that a triangle-closing walk resolves best.

- **RWR was my worst idea** (test MRR 0.7124, -0.0083 against DeepWalk). The hypothesis — that a personalised context sharpens pairwise proximity — is falsified. With a fixed length-40 budget, teleporting home 15% of the time spends about six tokens per walk re-emitting **start** and truncates the walk’s effective reach, so the corpus contains fewer distinct co-occurrences for the same cost. The breakdown localises the damage precisely: degree 1

is the *only* bucket where RWR beats DeepWalk (0.4499 vs 0.4386 test Hit@1, the largest degree-1 gain of any strategy — a short leash is exactly right when the source has one friend), and it is worse at every degree ≥ 2 , by -0.0218 in the ≥ 16 bucket.

- **Degree-corrected sampling also failed** (test MRR 0.7121, -0.0086) — and failed in an informative way. The hypothesis was that suppressing hubs would lift the *low*-degree buckets; the measurement says the opposite happened. It helped the high-degree end ($+0.0099$ at 8–15, $+0.0019$ at ≥ 16) and did its worst damage at 2–3 (-0.0199). Hubs are not noise to be corrected away: on a social graph a shared popular contact is real evidence that two users belong together, and re-weighting by $\deg(x)^{-1}$ discards exactly that evidence — which hurts most for the sparse users who had little other evidence to begin with.

Cold start is the real bottleneck, and no walk strategy can touch it. The single largest effect in this study is not a strategy at all. Test Hit@1 runs from 0.03–0.04 at source degree 0 to 0.68–0.69 at degree 8–15 — a gap of about 0.64, roughly *thirty-two* times the 0.0198 that separates the best strategy from the worst on that same metric. The 666 isolated users have no incident edge in G_{train} , so `sample_walks` correctly returns [] for them and their embedding row is zero *by construction*. The 216 test queries with such a source are therefore being scored from destination-side features alone, and the numbers confirm it exactly: the cold bucket’s seed-averaged Hit@1/MRR (0.0447/0.1887 for DeepWalk) match the random- Z control (0.0671/0.2009) and chance (0.0476/0.1736) rather than any learned result. Across all 21 runs that bucket wanders between 0.0139 and 0.1065 with no strategy stable across its own three seeds — DeepWalk alone scores 0.0833, 0.0231, 0.0278. What little ordering survives comes from a candidate-side popularity prior rather than from anything the source’s embedding contributes; Task 4 measures this directly and finds none of the 216 queries fully tied (median 20 distinct scores of 21), so the run-to-run scatter is genuine instability, not tie-breaking. Either way, reading the cold-start column as evidence about a sampling strategy would be a mistake. Fixing it requires information from outside the observed social graph — structural or profile features, or the user–artist edges used in Task 4 — not a better walk.

The non-monotone top bucket. Hit@1 peaks at degree 8–15 and falls back for ≥ 16 (seed-averaged 0.6904 $\rightarrow$ 0.6502 for triadic). More friends means more embedding evidence but also a harder ranking problem: a high-degree user is plausibly connected to many of the 20 sampled negatives, so the single held-out positive is genuinely harder to place first. MRR falls much less than Hit@1 (0.7959 $\rightarrow$ 0.7744), consistent with the positive still being ranked near the top rather than lost. These two top buckets are also where triadic beats DeepWalk by the widest margin ($+0.0121$ and $+0.0125$ Hit@1, against $+0.0038$ at degree 1), which fits the same story: for a hub user, knowing *which* community a candidate belongs to is what separates the true friend from a plausible negative.

Practical conclusion. If I had to ship one embedding for both products, it would be the triadic walk: it is the best on both tasks, it stays first-order, and it needs no (p, q) tuning. It is not free — sampling the whole 69,580-walk corpus takes 1.9s against DeepWalk’s 0.7s, because each step draws from a precomputed weighted table rather than a plain list — but 1.9s is negligible next to the ≈ 40 s of Skip-Gram training that follows it, so the choice costs nothing that matters in practice. But the honest headline is that on this graph the choice of random-walk strategy moves test MRR by at most 1.3 points, while the degree of the query user moves it by about 60 (from

0.19 in the cold bucket to 0.80 at degree 8–15); effort is better spent on the cold-start population than on the sampler.

2 Task 4 — Heterogeneous Graph Embedding (optional practice)

2.1 Method

The graph. H_{train} is the notebook’s `add_artists(G_train)`, used verbatim: 15,466 nodes (7,624 users + 7,842 artists) and $19,464 + 3,014,361 = 3,033,825$ edges. Every figure in the provided statistics file is reproduced exactly (7,451 users like ≥ 1 artist, 173 like none; artists per user median 400, max 944 — matching the task description’s own table). The artist layer is $155\times$ denser than the social layer, and that fact drives every result below.

What is reused unchanged. This is what makes the comparison in Table 4 legitimate, so I state it precisely. `train_embedding`, `LinkPredictor` and `CountryClassifier` are *the same Python objects* measured in Task 3: my sampler is a subclass of Task 3’s `WalkEmbedding` that overrides `sample_walks` and nothing else, and Task 3’s module is imported byte-identically (verified by `sha256`). The per-node budget (10 walks of length *up to* 40) and every Skip-Gram hyper-parameter ($d=128$, window 5, 5 negatives, 5 epochs, `workers=1` for bit-exact reproducibility) are unchanged. Evaluation is `run_experiment` with H_{train} and $N_{\text{users}} + N_{\text{artists}}$, slicing Z to $[: N_{\text{users}}]$; degree buckets always come from the *social* graph G_{train} . **The sampler and the graph are the only things that change.**

Schema convention. A schema is a symmetric type string whose repeating cycle is `schema[:-1]`: U-A-U gives UAUUA..., U-U-A-U gives UUAUUA..., U-U-U-A-U gives UUUUA... At each step the next node is drawn uniformly from the neighbours of the demanded type. Since U-A-U-U is a cyclic rotation of U-U-A-U and yields the same multiset of step types, I run one schema per rotation class. The three then form a one-parameter family indexed by the fraction of steps forced to be social (0, $1/3$, $1/2$, with Task 3’s DeepWalk as the 1 endpoint), the axis of Figure 4.

Start nodes, and dead ends. `embed_graph` starts walks from *every* node of H , artists included. I rotate the cycle to the start node’s type rather than discarding artist starts, so an artist begins AUUAU... — the same meta-path read from the other end; artists thus get trained vectors that serve as shared context, and are sliced off before evaluation. A walk *stops* whenever the type the schema demands next is absent. At the *first* step this drops the walk entirely (a user with no artist under U-A-U, 173 of them; a socially isolated user under U-U-A-U/U-U-U-A-U, 666), exactly as Task 3’s sampler returns [] for an isolated node — so those users keep an all-zero row in *both* tasks and the cold-start comparison stays like-for-like. But it also fires *mid-walk*, overwhelmingly when an $A \rightarrow U$ step lands on one of the 666 isolated users and the schema then asks for $U \rightarrow U$. That case is the common one, and it is why walk count and corpus size are different quantities: U-A-U emits 152,930 walks from 15,293 starts, *all* of full length 40, whereas U-U-A-U and U-U-U-A-U emit 148,000 from 14,800 starts with mean length only 24.3 and 27.5 (32.2% and 43.7% reach 40). Terminating on a dead end is standard metapath2vec behaviour, but Section 2.2 shows its effect must be measured, not assumed.

One idea of my own: rare-artist sampling. A uniform $U \rightarrow A$ step lands on a popular artist far more often than a niche one, and shared taste for a top-100 artist says almost nothing about

two users. I therefore add an optional weighting in which a step *into* an artist is drawn $\propto \deg(a)^{-\alpha}$ (steps into a user stay uniform). At $\alpha=1$ the median popularity of the artist a walk visits falls from 613 to 204.5 — essentially the dataset median (205), i.e. the bias towards hubs is removed.

```
class MetaPathWalkEmbedding(WalkEmbedding): # Task 3's class, one method overridden

    def __init__(self, schema="UAU", alpha=None, seed=RANDOM_SEED, **params):
        super().__init__(seed=seed, **params)
        assert schema[0] == schema[-1] # schema must be symmetric
        self.schema, self.cycle, self.alpha = schema, schema[:-1], alpha

    def sample_walks(self, G, start):
        tables, ntype = self._typed_tables(G) # node -> {type: (nbrs, cum_weights)}
        t0 = ntype[start]
        if t0 not in self.cycle:
            return []
        k = self.cycle.index(t0)
        rot = self.cycle[k:] + self.cycle[:k] # rotate the cycle to the start's type
        m, L, rng = len(rot), self.walk_length, self._rng

        walks = []
        for _ in range(self.num_walks):
            walk, i = [start], 0
            while len(walk) < L:
                entry = tables[walk[-1]].get(rot[(i + 1) % m])
                if entry is None: # no neighbour of the required type
                    break
                nbrs, cum = entry
                walk.append(rng.choice(nbrs) if cum is None # uniform
                           else rng.choices(nbrs, cum_weights=cum, k=1)[0]) # rare-artist
                i += 1
            if len(walk) > 1:
                walks.append(walk)
        return walks
```

Listing 5: The meta-path sampler. Everything else — training, link predictor, country classifier — is unchanged from Task 3.

`_typed_tables` is built once per graph from the notebook’s `typed_adjacency(G)` and cached: per node, the neighbour *list* per type plus cumulative $\deg(a)^{-\alpha}$ weights when rare-artist sampling is on. A step is therefore $O(1)$ (or $O(\log d)$ weighted) and the whole corpus takes 2–8s, against 60–115s of Skip-Gram training.

2.2 Protocol, and the controls the comparison rests on

Every configuration is run at seeds $\{0, 1, 2\}$ and all reported numbers are 3-seed means; the “noise floor” row of Table 4 is the mean within-configuration standard deviation, which is what licenses calling a gap real.

A confound must be removed first. Task 3 walks from the 6,958 non-isolated *users*; Task 4 walks from all 15,466 nodes of *H*, so a naive reading would credit the artist structure with what is really just more training data. The quantity to equalise is *not* the walk count but the **token** count — Skip-Gram consumes tokens, and by the dead-end argument above a meta-path walk is often far shorter than 40. Measured over all start nodes, the 10-walk schemas emit: U-A-U 152,930 walks all of full length, 6,117,200 tokens; U-U-A-U 148,000 walks of mean length 24.3, 3,593,757 tokens; U-U-U-A-U 148,000 walks of mean length 27.5, **4,068,117** tokens. The social-graph controls emit

full-length walks throughout: $\times 10$, 2,783,200 tokens; $\times 15$, **4,174,800**; $\times 22$, 6,123,040.

So U-A-U really does match a 22-walk DeepWalk (-0.1% tokens), but U-U-U-A-U — the row that carries the conclusion — emits a third fewer tokens at a similar walk count, so matching on walks alone would hand the control 50% more data than the schema it is meant to adjudicate. I therefore report **two** controls: DeepWalk $\times 22$ (walk-matched) and DeepWalk $\times 15$ (**token-matched** to U-U-U-A-U at $+2.6\%$, so the residual still favours the control). The token-matched one is the fair comparator and is the reference column of Table 5 and both figures. Corpus size alone ($\times 10 \rightarrow \times 15$) buys $+0.0109$ test Hit@1 and $+0.0062$ test MRR, but nothing on country micro-F1 (-0.0027).

2.3 Results

strategy	link prediction				country classification			
	val H@1	val MRR	test H@1	test MRR	val mi-F1	val ma-F1	test mi-F1	test ma-F1
<i>Task 3 — social graph G, 10 walks/node</i>								
DeepWalk (uniform)	0.6040	0.7227	0.6009	0.7207	0.7641	0.6639	0.7629	0.6833
node2vec $p=1, q=0.5$	0.5995	0.7202	0.5959	0.7184	0.7636	0.6621	0.7621	0.6825
node2vec $p=1, q=2$	0.6027	0.7210	0.6011	0.7196	0.7619	0.6467	0.7607	0.6802
node2vec $p=q=0.25$	0.6039	0.7217	0.6061	0.7228	0.7610	0.6581	0.7628	0.6796
RWR $r=0.15$	0.5870	0.7119	0.5889	0.7124	0.7637	0.6521	0.7620	0.6733
deg-corrected $\alpha=1$	0.6003	0.7144	0.5990	0.7121	0.7575	0.6421	0.7607	0.6781
triadic	0.6066	0.7247	0.6087	0.7248	0.7645	0.6833	0.7661	0.6974
<i>Budget controls — social graph G, larger walk budget</i>								
DeepWalk $\times 15$, token-matched	0.6110	0.7267	0.6118	0.7269	0.7650	0.6678	0.7601	0.6764
DeepWalk $\times 22$, walk-matched	0.6170	0.7298	0.6179	0.7292	0.7623	0.6643	0.7624	0.6852
<i>Task 4 — heterogeneous graph H, 10 walks/node</i>								
type-blind DeepWalk on H	0.4648	0.6306	0.4534	0.6229	0.6898	0.4111	0.6804	0.3794
metapath U-A-U	0.4201	0.5906	0.4133	0.5849	0.6457	0.3630	0.6489	0.3516
metapath U-U-A-U	0.5661	0.6994	0.5525	0.6876	0.7624	0.5738	0.7632	0.5675
metapath U-U-U-A-U	0.6215	0.7428	0.6163	0.7361	0.7956	0.6337	0.7887	0.6353
U-A-U rare-artist $\alpha=1$	0.4270	0.5954	0.4261	0.5942	0.7038	0.4245	0.6857	0.4007
<i>noise floor (mean sd)</i>	0.0038	0.0029	0.0050	0.0032	0.0072	0.0134	0.0038	0.0060

Table 4: The notebook’s `comparison_table()` (plus a validation macro-F1 column, which `run_experiment` does not record), Task 3 and Task 4 together; 3-seed means, Task 3 rows measured in Task 3 and not re-run. Noise floor = mean within-configuration sd over all 14 rows. Validation columns are where `fit` selects C , hence optimistic; the argument rests on the test columns.

2.4 Analysis

Aggregate link prediction: no, the artists do not help. Against the *token-matched* control, U-U-U-A-U reaches test Hit@1 0.6163 versus 0.6118: $+0.0045$, or $+0.9$ noise sd — nothing. Against the walk-matched control the same comparison is 0.6163 versus 0.6179, i.e. -0.0016 (-0.3 sd). The useful observation is that **the sign of the Hit@1 gap depends on which control you pick and its size never reaches one noise sd either way**, which is a null result far more robustly than any single control could establish. Test MRR is the one place a real edge survives: $+0.0093$ (2.9 sd) against the token-matched control, $+0.0070$ (2.2 sd) against the walk-matched one — consistent, but small. Compared against the *unmatched* 10-walk DeepWalk the schema looks like a clean $+0.0154$ Hit@1 win, and roughly two-thirds of that ($+0.0109$) is bought by corpus size

source degree	n	DeepWalk $\times 15$		U-A-U		U-U-A-U		U-U-U-A-U	
		H@1	MRR	H@1	MRR	H@1	MRR	H@1	MRR
0 (cold)	216	0.0803	0.2210	0.3596	0.5320	0.2994	0.4712	0.2824	0.4666
1	266	0.4361	0.5642	0.4098	0.5702	0.4273	0.5924	0.5063	0.6338
2-3	469	0.5814	0.6900	0.4357	0.6026	0.5153	0.6548	0.5465	0.6744
4-7	842	0.6690	0.7734	0.4406	0.6083	0.6065	0.7268	0.6496	0.7606
8-15	939	0.6887	0.7940	0.4441	0.6092	0.6145	0.7371	0.6894	0.7932
≥ 16	1439	0.6504	0.7739	0.3785	0.5601	0.5536	0.6931	0.6423	0.7641

Table 5: Test-split link prediction by the source user’s degree *in the social graph* G_{train} , 3-seed means. The reference column is the *token-matched* control. The cold row and every other row point in opposite directions.

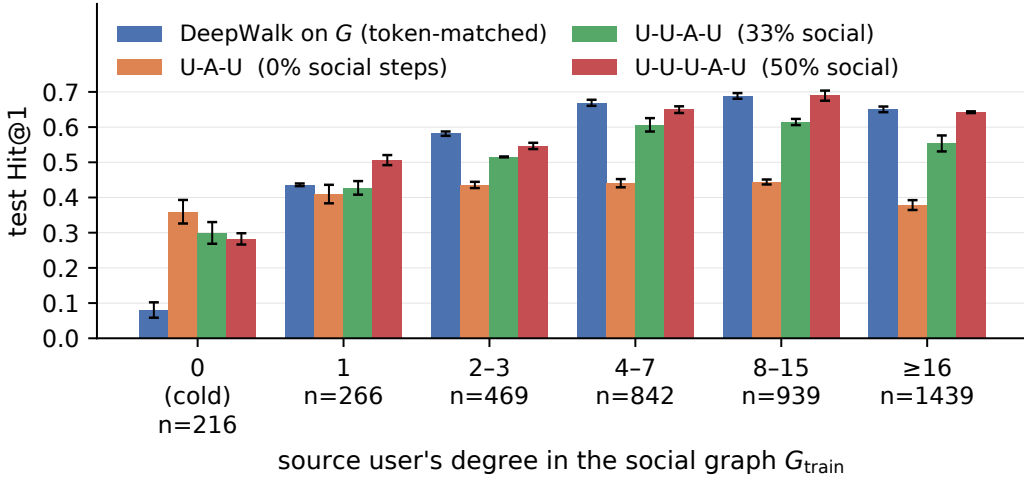


Figure 3: Test Hit@1 by social degree of the source user; error bars ± 1 sd over three seeds. Adding artists is not a uniform gain or loss — it transfers accuracy from well-connected users to cold-start ones.

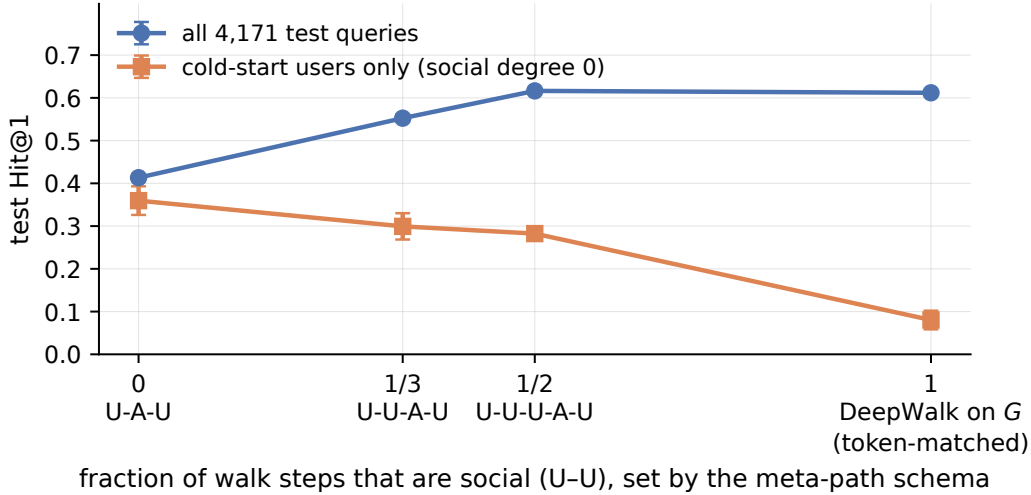


Figure 4: The same three schemas plus the token-matched DeepWalk, by the fraction of walk steps forced to be social. The two populations move in opposite directions; the schema is the knob that trades one against the other.

alone. This is why the controls were worth running: without them the honest answer inverts into a false positive.

Cold-start users: yes, dramatically, and this is not a budget artefact. 666 users have no friend in G_{train} and receive an all-zero row from any purely social walk; 216 test queries have such a source. Their test Hit@1 is 0.0803 for the token-matched control, and 0.0447/0.0494 for DeepWalk $\times 10/\times 22$ — the extra corpus buys them essentially nothing, because no amount of walking reaches a node with no edges. They are not ranked at random, though: with $u = 0$ the pair features $|u - v|$ and $\|u - v\|$ reduce to $|v|$ and $\|v\|$, so the predictor still separates candidates on candidate-only features (0 of the 216 queries have fully tied scores; median 20 distinct of 21). That floor is a weak popularity prior, not chance. But 647 of those 666 users (97.1%) like at least one artist, with a median of 334 artists each, and 209 of the 216 cold test sources (96.8%) are covered. Through the artist layer they become reachable, and their Hit@1 rises to 0.2824 under U-U-U-A-U and 0.3596 under U-A-U — respectively $3.5\times$ and $4.5\times$ the control, gaps of 7.5 and 10.3 times the 0.0271 mean seed spread of this bucket (Figure 3). This is the one place where the heterogeneous structure does something no amount of social-graph effort can.

Signal or noise? Both, and density decides which. A U-A-U step connects two users who share *any* artist. Sampling 40 users, the median number of others reachable in one such step is 7,307 of 7,623 — **about 96% of the graph** — against a median social 2-hop neighbourhood of 61. Such an edge is therefore almost devoid of information: it is not far from a link to a uniformly random user. That explains the whole shape of Table 5. For a user with ≥ 16 friends the social graph already supplies a sharp, small candidate set, and diluting it with a near-complete graph costs $0.6504 \rightarrow 0.3785$ Hit@1. For a user with 0 friends the alternative is not a sharper signal but an all-zero vector ranked on a popularity prior, so even a weak ordering (the layer is near-complete but not uniform — how *many* artists two users share still varies) is worth a great deal. Same edges, opposite value, depending entirely on what the user already had.

The meta-path constraint is doing real work. A type-blind DeepWalk on H takes a social step only 0.72% of the time (over 31,122 sampled steps), because a user has ~ 395 artist neighbours and ~ 5 social ones; it is *de facto* U-A-U. Yet it beats explicit U-A-U on every column (+0.0401 test Hit@1). Those rare social steps are worth a great deal — precisely the `metapath2vec` argument: on a graph this lopsided, leaving the transition rule to the degree distribution discards the sparse, informative edge type, and the schema is how you take that choice back. The stronger form is the monotone family in Figure 4: as the forced social fraction rises $0 \rightarrow 1/3 \rightarrow 1/2$, overall test Hit@1 goes $0.4133 \rightarrow 0.5525 \rightarrow 0.6163$ while the cold bucket falls $0.3596 \rightarrow 0.2994 \rightarrow 0.2824$, without collapsing to the 0.0803 of a pure social walk — a socially isolated user can never *start* a U-U-U-A-U walk, but the $A \rightarrow U$ step lets other walks reach them. Truncation shrinks the corpus of exactly the higher-social schemas, so it works *against* the rising trend, which is therefore safe from the budget objection.

Country classification: helped on micro-F1, hurt on macro-F1. Against the token-matched control, U-U-U-A-U gains +0.0286 test micro-F1 (0.7887 vs 0.7601, 7.6 sd) and loses -0.0411 test macro-F1 (0.6353 vs 0.6764, 6.9 sd); against the walk-matched control the same two gaps are +0.0263 and -0.0498 , so neither depends on the control. Both are far outside the noise, and the split between them is interpretable: music taste is regional, so artists carry genuine country signal and lift the large classes micro-F1 measures — but that signal is coarse, regional rather than national, while macro-F1 is dominated by the small countries, which in the social graph are tight, well-separated communities that the near-complete artist layer blurs together. Task 3’s triadic walk remains the study’s best macro-F1 model (0.6974); no heterogeneous variant comes close.

Rare-artist sampling works, and confirms the diagnosis. Weighting $U \rightarrow A$ by $\deg(a)^{-1}$ improves plain U-A-U on all eight columns: +0.0129 test Hit@1 (2.6 sd), +0.0094 test MRR (2.9 sd), +0.0368 test micro-F1 (9.8 sd) and +0.0491 test macro-F1 (8.2 sd). Since the only change is which artist a walk passes through, this is direct evidence that the *popular* artists specifically inject the noise, and that the density problem is partly fixable inside the sampler. It does not rescue U-A-U to competitiveness — the schema still lacks the social edges that matter most — but it is the largest single improvement I obtained over a baseline schema.

2.5 Conclusion

Heterogeneous structure helped *one* of the two downstream tasks and one sub-population, and hurt elsewhere. It did essentially nothing for aggregate friend recommendation once the corpus was matched *in tokens*; it improved country micro-F1 by a wide margin and degraded macro-F1 by a comparable one; and it lifted cold-start friend recommendation by 3.5–4.5 $\times$ over a budget-matched baseline, which is the result worth keeping. The recommendation that follows is not a single best schema but a split one: a social-graph walk (Task 3’s triadic) for users who have friends, and a U-A-U walk — ideally with rare-artist weighting — for the 9% who do not, the crossover sitting between degree 0 and 1 (Table 5). The broader lesson: on a graph where one edge type is 155 $\times$ denser than the other, the meta-path is not a modelling flourish but the only thing standing between the walk and a near-complete graph.